732A94 Advanced R programming

Lecture 1

Johan Alenlöv (Slides based on Leif Jonssons, Måns Magnusson and Krzystof Bartoszek)

Linköpings Universitet

Lecture 1:

- About the course
- Presentation
- Course practicals
- Why R?
- Introduction to R-programming
    - Data structures
    - Logic and sets
    - Subsetting/filtering
    - Functions

Aim of the course

- Write R programs and packages
- Write performant code
- Learn basic software engineering practices

Aim of the course

- Write R programs and packages
- Write performant code
- Learn basic software engineering practices
- Will be (one of) your preimary tools for the next two years.

Course plan

Part 1: R syntax

Period: Week 1-2

Students work: individually

Lab: Documented R file

Computer lab

Topics

- Basic R syntax
- Basic data structures
- Program control
- R packages

Course plan

Part 2: Advanced topics

Period: Week 3-7

Students work: In groups

Turn in: R packages on GitHub Seminar: **OBLIGATORY**

1 Lab to help with GitHub

Topics

- Performant code:
  - Write quality code
  - Writing fast code
- Linear algebra, object orientation, graphics
- Advanced I/O
- Computational complexity

Teaching staff

- Johan Alenlöv
  - Examiner, lectures
- Bayu and more
  - Labs
  - Seminars

Evaluation of 2025 course

Changes for this year

- New teacher/examiner

Course Practicals

- Course code: 732A94
- https://www.ida.liu.se/~732A94/
- https://github.com/STIMALiU/AdvRCourse
- LISAM submission

Course literature

There are a bunch of books listed on the course webpage, they should all be available online through the library

Also a lot of resources available through google

Examination

Weekly mandatory labs/projects/seminars

Deadline: After corresponding lecture and seminar (for labs 3-6) stated on lab/Lisam

Obligatory presentation and seminar attendance

Computer exam (on campus linux computers)

A:[18,20], B:[16,18), C:[14,16), D:[12,14), E:[10,12), F:[0,10)

Bonus points

Bonus points

Computational complexity session bonus points can be obtained by correctly solving the exercises. Solutions to the exercises should be brought to the session. Then, students (from those that brought their solutions) will be selected at random for each exercise to present their solutions. Failure to present a correct solution will result in reduced bonus points from this session. In particular this implies that the student who wants the bonus point has to be present at the session.

Why R?

Choose the right tool for the job!

Your main job will be statistics and data analysis....

R is (nearly always) the right tool for that job!

Pros

- Popular (among statisticians)
- Good graphics support
- Open source – available on all major platforms!
- High-level language – focus on data analysis
- Strong community – vast amount of packages
- Powerful for communicating results
- API’s to high-performance languages as C/C++ and Java

Cons

- “Ad hoc”, complex, language
- Can be sloooow
- Can be memory inefficient
- Hard’ish to troubleshoot
- Inferior IDE support compared to state of the art

Use arrow for assignments

```
a <- 1
```

We have a console where we can type commands and use like a calculator

```
2+3
```

```
## [1] 5
```

```
2/3
```

```
## [1] 0.6666667
```

```
2**4
```

```
## [1] 16
```

Variable types

Variable type	Short	typeof()	R example
Boolean	logi	logical	`TRUE`
Integer	int	integer	`1L`
Real	num	double	`1.2`
Complex	cplx	complex	`0+1i`
Character	chr	character	`"I <3 R"`

factor: enumerated/factor variable, `levels()`: permitted values

```
as.factor(c("a","a","b","c"))
```

```
## [1] a a b c
## Levels: a b c
```

Data structures

Dimension	Homogeneous data	Heterogeneous data
1	`vector`	`list`
2	`matrix`	`data.frame`
n	`array`	

- constructors `vector()`, `list()`
- Name dimensions: ‘dimnames()“‘

Arithmetic

- Vectorized operations (element wise)
- Recycling
- Statistical functions

See reference card...

Logic operations

In symbols	A	B	$\neg A$	$A \wedge B$	$A \vee B$
In R	A	B	!A	A&B	A|B
	TRUE	FALSE	?	?	?
	TRUE	TRUE	?	?	?
	FALSE	FALSE	?	?	?
	FALSE	TRUE	?	?	?

Logic operations

In symbols	A	B	$\neg A$	$A \wedge B$	$A \vee B$
In R	A	B	!A	A&B	A|B
	TRUE	FALSE	FALSE	FALSE	TRUE
	TRUE	TRUE	FALSE	TRUE	TRUE
	FALSE	FALSE	TRUE	FALSE	FALSE
	FALSE	TRUE	TRUE	FALSE	TRUE

Logic operations

In symbols	$\wedge_{i=1}^{n} a_i$	$\vee_{i=1}^{n} a_i$	$\{j : a_j == \mathrm{TRUE}\}$
In R	`all(A)`	`any(A)`	`which(A)`

Relational operators

In symbols	$a < b$	$a \leq b$	$a \neq b$	$a = b$	$a \in A$
In R	`a<b`	`a<=b`	`a != b`	`a == b`	`a %in% A`

Comparing can be tricky

```
x <- sqrt(2)
x*x
```

```
## [1] 2.0000000000000000444089
```

```
x*x == 2
```

```
## [1] FALSE
```

```
all.equal(x*x, 2)
```

```
## [1] TRUE
```

```
identical(2L, 2)
```

```
## [1] FALSE
```

```
identical(2L, 2L)
```

```
## [1] TRUE
```

Vectors: use []

- Index by:
  - Positive integers include
  - Negative integers exclude
  - Logical vector includes TRUE

```
vect <- c(6, 7, 8, 9)
vect[vect>7]; vect[which(vect > 7)]
```

```
## [1] 8 9
```

```
## [1] 8 9
```

```
vect[c(1,2)]
```

```
## [1] 6 7
```

```
vect[c(-1,-2)]
```

```
## [1] 8 9
```

Matrices

- Index by ‘[i,j]‘
- Two dimensions
- Can reduce to vector
- Column-major storage in memory
    - Consider how access will take place, use option `matrix(...,byrow=)`

Matrices

```
mat <- matrix(c(1,2,3,4,5,6), nrow = 2)
mat
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
mat[c(1,2),c(1,3)]
```

```
##      [,1] [,2]
## [1,]    1    5
## [2,]    2    6
```

```
mat[mat>4]
```

```
## [1] 5 6
```

Lists

- Use `[ ]` to access list elements
- use `[[ ]]` to access list content
- Indexed like vectors
- Use `$` to access list element by name
- Not like a typical list in other languages
- General container, can contain anything

Lists

```
lst <- list(a=47, b = 11)
lst
```

```
## $a
## [1] 47
##
## $b
## [1] 11
```

```
lst[1]
```

```
## $a
## [1] 47
```

```
lst[[1]]
```

```
## [1] 47
```

Lists cont.

```
lst$b
```

```
## [1] 11
```

```
x <- "a";lst[which(names(lst)==x)]
```

```
## $a
## [1] 47
```

```
lst[[which(names(lst)==x)]];lst[[x]]
```

```
## [1] 47
```

```
## [1] 47
```

Data frames

- Very powerful data structure
- Can roughly think about it as the R representation of a CSV
- Can be loaded from a CSV
- Can be accessed as both a matrix and a list

Data.frame

```
df <- data.frame(a = c(1,2), b = c(3,4))
df$a
```

```
## [1] 1 2
```

```
df[1,c(1,2)]
```

```
##   a b
## 1 1 3
```

Assigning subsets

- Change values in data structures
- Works for all mentioned data types
- Use `<-` to set values

Assigning subsets

```
mat
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
mat[mat>4] <- 100
mat
```

```
##      [,1] [,2] [,3]
## [1,]    1    3  100
## [2,]    2    4  100
```

All combinations

```
expand.grid(v1 = 1:3, v2 = c("a","b","c"))
```

```
##   v1 v2
## 1  1  a
## 2  2  a
## 3  3  a
## 4  1  b
## 5  2  b
## 6  3  b
## 7  1  c
## 8  2  c
## 9  3  c
```

Functions

```r
my_function_name <- function(x,y) {
  z <- x^2 + y^2
  return(z)
}
```

Unlike many other languages return in R is a **function**. In other languages `return` is usually a reserved word.

By default R returns the last computed value of the function, so return is not strictly necessary in simple cases.

Help I’m stuck!

Access help from the documentation

```
# Find help for specific function
?function_name
help(function_name)
# Serach help for word
??word

#Example
?"-"
help("+")
```

markmyassignment (Labs 1 and 2)

R package for automatic marking of R assignments for students and teachers based on `testthat` test suites. Developed by Måns Magnusson, Oscar Pettersson and Jonas Wallin.

https://github.com/MansMeg/markmyassignment

Test-driven development (TDD): Software requirements are made into test cases, before software is finished. As the software is developed, all software is tested against all the test cases.

End of Lecture

Questions?

See you tomorrow!